

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web 2

ENTREGA 1B

shadcn/ui + Estructura + Base de Datos

Proyecto: gestión-académica

Stack: Next.js 16 · Supabase Cloud · shadcn/ui · TypeScript

Fase 1 — Fundación | Prerrequisito: Entrega 1A completada

Abril 2026

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Instalar y configurar shadcn/ui en un proyecto Next.js 16 existente.
- Comprender cómo shadcn/ui genera componentes locales (no dependencias externas).
- Crear la estructura completa de carpetas del proyecto según la arquitectura definida.
- Ejecutar un esquema SQL en Supabase Dashboard para crear las tablas de la aplicación.
- Instalar las dependencias adicionales del proyecto (Supabase, Zod, React Hook Form).

Prerrequisitos

Debes haber completado la Entrega 1A. Tu proyecto debe tener:

- ✓ Proyecto **gestion-academica** creado con Next.js 16
- ✓ Archivo **.env.local** con las credenciales de Supabase
- ✓ `npm run dev` funcionando en `http://localhost:3000`

Instalar shadcn/ui

shadcn/ui no es una librería tradicional que se instala como dependencia. En su lugar, es un CLI que copia componentes directamente a tu proyecto como código fuente. Esto te da control total sobre cada componente.

Detiene el servidor de desarrollo (**Ctrl + C**) y ejecuta el siguiente comando en la raíz del proyecto:

Terminal

```
npx shadcn@latest init
```

El CLI detectará que tienes un proyecto Next.js y te hará una pregunta sobre el color base:

Salida esperada

```
✓ Preflight checks.
✓ Verifying framework. Found Next.js.
✓ Validating Tailwind CSS config. Found v4.
✓ Validating import alias.
✓ Which color would you like to use as the base color? > Neutral
✓ Writing components.json.
✓ Checking registry.
✓ Updating CSS variables in app/globals.css
✓ Installing dependencies.
✓ Created 1 file:
  - lib/Utils.ts
```

```
Success! Project initialization completed.
```

You may now add components.

Nota: Color base

Elige Neutral como color base. Esto genera una paleta profesional y neutra que funciona bien para aplicaciones empresariales. Puedes personalizar los colores más adelante.

¿Qué hizo el CLI?

El comando `init` realizó tres acciones importantes:

1. **Creó `components.json`** — archivo de configuración que define dónde se guardan los componentes, qué alias usar, y el estilo base.
2. **Modificó `app/globals.css`** — agregó variables CSS para colores del tema (light/dark mode), radio de bordes, y otros tokens de diseño.
3. **Creó `lib/utils.ts`** — función utilitaria `cn()` que combina clases de Tailwind de manera inteligente.

Archivo: `lib/utils.ts`

Este archivo contiene la función `cn()` que usarás en toda la aplicación:

```
Archivo: lib/utils.ts (generado automáticamente)
import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}
```

La función **`cn()`** combina `clsx` (para clases condicionales) con `twMerge` (para resolver conflictos entre clases de Tailwind). Por ejemplo: `cn("px-4 py-2", isActive && "bg-blue-500")`.

Instalar los primeros componentes de shadcn/ui

Vamos a instalar los componentes que necesitaremos en la Fase 1. Cada comando copia el código del componente a tu carpeta `components/ui/`:

```
Terminal
# Componentes esenciales para Fase 1
npx shadcn@latest add button
npx shadcn@latest add input
npx shadcn@latest add label
npx shadcn@latest add card
npx shadcn@latest add sonner
```

Tip: Instalar múltiples componentes

Puedes instalar varios componentes en un solo comando: `npx shadcn@latest add button input label card sonner`

Después de la instalación, tu carpeta `components/ui/` contendrá:

Estructura generada

```

components/
└─ ui/
   └─ button.tsx      # Botón con variantes (default, destructive, outline, etc.)
   └─ card.tsx        # Contenedor con header, content, footer
   └─ input.tsx       # Campo de texto estilizado
   └─ label.tsx       # Etiqueta para formularios
   └─ sonner.tsx      # Notificaciones toast

```

Verificar que los componentes funcionan

Abre **app/page.tsx** y reemplaza su contenido con:

Archivo: app/page.tsx

```

import { Button } from "@/components/ui/button"

export default function Home() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-center gap-4">
      <h1 className="text-4xl font-bold">Gestión Académica</h1>
      <p className="text-muted-foreground">
        Sistema de gestión universitaria
      </p>
      <div className="flex gap-2">
        <Button>Botón Default</Button>
        <Button variant="outline">Outline</Button>
        <Button variant="destructive">Destructive</Button>
      </div>
    </main>
  )
}

```

Ejecuta **npm run dev** y visita <http://localhost:3000>. Deberías ver tres botones con estilos diferentes. Si los ves, shadcn/ui está funcionando correctamente.

Nota: ¿Qué es text-muted-foreground?

Las clases como text-muted-foreground son variables CSS que shadcn/ui agregó a tu globals.css. Se adaptan automáticamente al modo claro/oscuro sin que escribas código adicional.

Instalar dependencias del proyecto

Además de shadcn/ui, necesitamos instalar las librerías que usaremos a lo largo del proyecto:

Terminal

```

# Cliente de Supabase para browser y server
npm install @supabase/supabase-js @supabase/ssr

# Validación de formularios
npm install zod

# Manejo de formularios React
npm install react-hook-form @hookform/resolvers

```

Verifiquemos para qué sirve cada dependencia:

Paquete	Propósito
<code>@supabase/supabase-js</code>	Cliente oficial de Supabase. Permite interactuar con la BD, Auth y Storage.
<code>@supabase/ssr</code>	Adaptador para frameworks SSR. Maneja cookies de autenticación en servidor.
<code>zod</code>	Librería de validación de esquemas. Define reglas para formularios y API Routes.
<code>react-hook-form</code>	Manejo eficiente de formularios en React. Minimiza re-renders.
<code>@hookform/resolvers</code>	Conecta Zod con React Hook Form para validación automática.

Crear la estructura de carpetas del proyecto

La organización de carpetas es crítica en proyectos reales. Vamos a crear toda la estructura que usaremos en las 7 entregas. Esto nos permitirá navegar cómodamente desde el inicio.

Importante

Crea TODAS las carpetas ahora aunque estén vacías. Esto te da una visión clara de la arquitectura completa del proyecto desde el día uno.

Ejecuta los siguientes comandos en la raíz del proyecto (`gestion-academica/`):

```
Terminal – Parte 1: Rutas
# — Estructura de rutas de la aplicación —

# Grupo de autenticación (rutas públicas)
mkdir -p "app/(auth)/login"
mkdir -p "app/(auth)/registro"
mkdir -p "app/(auth)/auth/callback"

# Grupo del dashboard (rutas protegidas)
mkdir -p "app/(dashboard)/estudiante/materias"
mkdir -p "app/(dashboard)/estudiante/inscripciones"
mkdir -p "app/(dashboard)/estudiante/perfil"

mkdir -p "app/(dashboard)/docente/materias"
mkdir -p "app/(dashboard)/docente/perfil"

mkdir -p "app/(dashboard)/admin/usuarios"
mkdir -p "app/(dashboard)/admin/materias/nueva"
mkdir -p "app/(dashboard)/admin/inscripciones"

# API Routes
mkdir -p "app/api/auth/registro"
mkdir -p "app/api/auth/perfil"
mkdir -p "app/api/materias"
```

```
mkdir -p "app/api/inscripciones"
```

Terminal — Parte 2: Componentes y utilidades

```
# — Componentes organizados por dominio —
mkdir -p "components/auth"
mkdir -p "components/dashboard"
mkdir -p "components/materias"
mkdir -p "components/inscripciones"
mkdir -p "components/shared"

# — Librerías y utilidades —
mkdir -p "lib/supabase"
mkdir -p "lib/types"

# — Hooks personalizados —
mkdir -p "hooks"
```

Estructura resultante

Tu proyecto debe verse así (sin contar los archivos que aún no hemos creado):

Arquitectura completa del proyecto

```
gestion-academica/
├── app/
│   ├── (auth)/                                ← Grupo: rutas públicas
│   │   ├── login/
│   │   ├── registro/
│   │   └── auth/callback/
│   ├── (dashboard)/                          ← Grupo: rutas protegidas
│   │   ├── estudiante/
│   │   │   ├── materias/
│   │   │   ├── inscripciones/
│   │   │   └── perfil/
│   │   ├── docente/
│   │   │   ├── materias/
│   │   │   └── perfil/
│   │   └── admin/
│   │       ├── usuarios/
│   │       ├── materias/nueva/
│   │       └── inscripciones/
│   └── api/                                  ← API Routes (endpoints REST)
│       ├── auth/registro/
│       ├── auth/perfil/
│       ├── materias/
│       └── inscripciones/
├── components/
│   ├── ui/                                  ← shadcn/ui (auto-generados)
│   ├── auth/                              ← Formularios de login/registro
│   └── dashboard/                         ← Sidebar, header, stats
```

├─ materias/	← Cards, tablas, formularios
├─ inscripciones/	← Cards, tablas
├─ shared/	← Loading, empty states, errores
├─ lib/	
│ └─ supabase/	← Clientes Supabase (browser/server)
│ └─ types/	← Tipos TypeScript de la BD
│ └─ utils.ts	← cn() de shadcn + helpers
└─ hooks/	← Custom hooks de React

¿Qué son los grupos de rutas (paréntesis)?

En Next.js App Router, las carpetas entre paréntesis como **(auth)** y **(dashboard)** son **Route Groups**. Tienen dos características especiales:

4. **No afectan la URL:** la ruta `/login` apunta a `app/(auth)/login/page.tsx`, no a `app/auth/login/page.tsx`.
5. **Permiten layouts independientes:** cada grupo puede tener su propio `layout.tsx`. Así, `(auth)` tendrá un layout centrado para formularios, y `(dashboard)` tendrá un layout con sidebar.

Crear las tablas en Supabase Dashboard

Ahora vamos a crear las tablas de nuestra base de datos. Usaremos el Editor SQL de Supabase para ejecutar el esquema completo.

6. Abre tu proyecto en Supabase Dashboard (<https://supabase.com/dashboard>).
7. En el menú lateral, haz clic en **SQL Editor**.
8. Haz clic en **New query**.
9. Copia y pega el siguiente código SQL completo:

SQL – Parte 1: Tabla usuarios_perfil

```
-- =====
-- ESQUEMA: gestion-academica
-- Base de datos para sistema de gestión universitaria
-- =====

-- — TABLA 1: Perfiles de Usuario —
-- Vinculada a auth.users (autenticación de Supabase)
CREATE TABLE IF NOT EXISTS public.usuarios_perfil (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  nombre_completo TEXT NOT NULL,
  rol TEXT NOT NULL CHECK (rol IN ('estudiante', 'docente', 'admin')),
  carrera TEXT,
  telefono TEXT,
  avatar_url TEXT,
  activo BOOLEAN DEFAULT true NOT NULL,
  eliminado BOOLEAN DEFAULT false NOT NULL
```

```
);

-- Índices para búsquedas rápidas
CREATE INDEX idx_usuarios_rol ON public.usuarios_perfil(rol);
CREATE INDEX idx_usuarios_activo ON public.usuarios_perfil(activo);
```

SQL – Parte 2: Trigger updated_at

```
-- — FUNCIÓN: Actualizar updated_at automáticamente —
CREATE OR REPLACE FUNCTION public.actualizar_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_updated_at_usuarios
BEFORE UPDATE ON public.usuarios_perfil
FOR EACH ROW EXECUTE FUNCTION public.actualizar_updated_at();
```

SQL – Parte 3: Tabla materias

```
-- — TABLA 2: Materias —
CREATE TABLE IF NOT EXISTS public.materias (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    codigo TEXT UNIQUE NOT NULL,          -- Ej: "SIS-301"
    nombre TEXT NOT NULL,
    credits INT4 NOT NULL CHECK (credits BETWEEN 1 AND 10),
    docente_id UUID REFERENCES public.usuarios_perfil(id),
    semestre TEXT NOT NULL,              -- Ej: "2026-I"
    activa BOOLEAN DEFAULT true NOT NULL
);

CREATE INDEX idx_materias_docente ON public.materias(docente_id);
CREATE INDEX idx_materias_semestre ON public.materias(semestre);
```

SQL – Parte 4: Tabla inscripciones

```
-- — TABLA 3: Inscripciones —
CREATE TABLE IF NOT EXISTS public.inscripciones (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
    estudiante_id UUID NOT NULL REFERENCES public.usuarios_perfil(id),
    materia_id UUID NOT NULL REFERENCES public.materias(id),
    fecha_inscripcion DATE DEFAULT CURRENT_DATE NOT NULL,
    estado TEXT DEFAULT 'activa' NOT NULL
    CHECK (estado IN ('activa', 'retirada', 'completada')),
    UNIQUE(estudiante_id, materia_id)
);

CREATE INDEX idx_inscripciones_estudiante
ON public.inscripciones(estudiante_id);
```



```
CREATE INDEX idx_inscripciones_materia
ON public.inscripciones(materia_id);
```

SQL – Parte 5: Tabla calificaciones

```
-- — TABLA 4: Calificaciones (para uso post-MVP) —
CREATE TABLE IF NOT EXISTS public.calificaciones (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  inscripcion_id UUID NOT NULL REFERENCES public.inscripciones(id),
  tipo_evaluacion TEXT NOT NULL
    CHECK (tipo_evaluacion IN (
      'parcial_1', 'parcial_2', 'parcial_3',
      'final', 'practica', 'proyecto', 'extra'
    )),
  nota NUMERIC(4,1) NOT NULL CHECK (nota BETWEEN 0 AND 100),
  fecha DATE DEFAULT CURRENT_DATE NOT NULL,
  observaciones TEXT,
  registrado_por UUID NOT NULL REFERENCES public.usuarios_perfil(id),
  UNIQUE(inscripcion_id, tipo_evaluacion)
);

CREATE INDEX idx_calificaciones_inscripcion
ON public.calificaciones(inscripcion_id);

CREATE TRIGGER trigger_updated_at_calificaciones
BEFORE UPDATE ON public.calificaciones
FOR EACH ROW EXECUTE FUNCTION public.actualizar_updated_at();
```

SQL – Parte 6: Tabla documentos_academicos

```
-- — TABLA 5: Documentos Académicos (para uso post-MVP) —
CREATE TABLE IF NOT EXISTS public.documentos_academicos (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at TIMESTAMPTZ DEFAULT now() NOT NULL,
  titulo TEXT NOT NULL,
  descripcion TEXT,
  ruta_archivo TEXT NOT NULL,
  tipo_documento TEXT NOT NULL
    CHECK (tipo_documento IN (
      'syllabus', 'practica', 'examen',
      'material', 'tarea', 'otro'
    )),
  materia_id UUID NOT NULL REFERENCES public.materias(id),
  subido_por UUID NOT NULL REFERENCES public.usuarios_perfil(id),
  visible_estudiantes BOOLEAN DEFAULT false NOT NULL
);

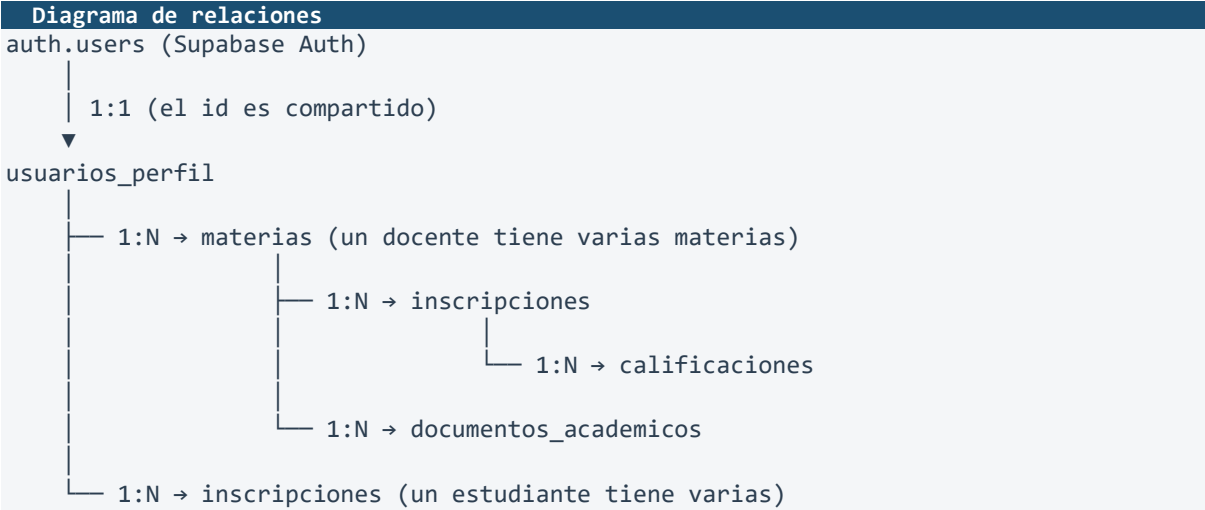
CREATE INDEX idx_documentos_materia
ON public.documentos_academicos(materia_id);
CREATE INDEX idx_documentos_subido_por
ON public.documentos_academicos(subido_por);
```

10. Haz clic en **Run** (o presiona Ctrl+Enter). Deberías ver: **Success. No rows returned.**

Tip: Verificar las tablas
Ve a Table Editor en el menú lateral de Supabase. Deberías ver las 5 tablas creadas: usuarios_perfil, materias, inscripciones, calificaciones, documentos_academicos.

Entender el esquema de base de datos

Antes de continuar, es fundamental entender la relación entre las tablas. Este diagrama muestra cómo se conectan:



Puntos clave del diseño:

Concepto	Explicación
UUID	Todas las tablas usan UUID como clave primaria (no números secuenciales). Son únicos globalmente y no revelan información sobre el orden de creación.
ON DELETE CASCADE	Si se elimina un usuario de auth.users, su perfil se elimina automáticamente. Mantiene la integridad referencial.
CHECK constraints	Validan datos a nivel de BD. Por ejemplo, rol solo puede ser 'estudiante', 'docente' o 'admin'. Esto es seguridad adicional a la del frontend.
UNIQUE constraints	Evitan duplicados. Un estudiante no puede inscribirse dos veces en la misma materia.
Soft delete	El campo 'eliminado' permite marcar registros como eliminados sin borrarlos físicamente. Útil para auditoría.
Índices	Aceleran las consultas frecuentes. Por ejemplo, idx_usuarios_rol hace rápidas las búsquedas por rol.
Trigger updated_at	Actualiza automáticamente la columna updated_at cada vez que se modifica un registro. No necesitas hacerlo manualmente.

Configurar next.config.ts

Abre el archivo **next.config.ts** y reemplázalo con la siguiente configuración:

```
Archivo: next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  // Dominios permitidos para imágenes externas
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "/*.supabase.co",
        pathname: "/storage/v1/object/public/**",
      },
    ],
  },
};

export default nextConfig;
```

Esta configuración permite que `next/image` cargue imágenes almacenadas en Supabase Storage (avatares de usuario, documentos, etc.).

Verificar la instalación completa

Vamos a crear una página de prueba que confirme que todas las dependencias están funcionando. Esta página la eliminaremos después.

Reemplaza el contenido de **app/page.tsx** con:

```
Archivo: app/page.tsx
import { Button } from "@components/ui/button"
import {
  Card,
  CardContent,
  CardDescription,
  CardHeader,
  CardTitle,
} from "@components/ui/card"

export default function Home() {
  return (
    <main className="flex min-h-screen items-center justify-center bg-background">
      <Card className="w-[400px]">
        <CardHeader>
          <CardTitle className="text-2xl">Gestión Académica</CardTitle>
          <CardDescription>
            Sistema de gestión universitaria — UPDS
          </CardDescription>
        </CardHeader>
```

```

    <CardContent className="space-y-4">
      <p className="text-sm text-muted-foreground">
        Stack: Next.js 16 · Supabase · shadcn/ui
      </p>
      <div className="flex gap-2">
        <Button>Iniciar Sesión</Button>
        <Button variant="outline">Registrarse</Button>
      </div>
      <p className="text-xs text-muted-foreground">
        Entrega 1B completada ✓
      </p>
    </CardContent>
  </Card>
</main>
)
}

```

Ejecuta **npm run dev** y visita <http://localhost:3000>. Deberías ver una tarjeta (Card) centrada con título, descripción y dos botones estilizados. Los botones aún no hacen nada — les daremos funcionalidad en la Entrega 2.

Verificación Final

Antes de continuar con la Entrega 2A, confirma que todos estos puntos se cumplen:

- ✓ `npx shadcn@latest init` se ejecutó sin errores
- ✓ La carpeta `components/ui/` contiene `button.tsx`, `card.tsx`, `input.tsx`, `label.tsx`, `sonner.tsx`
- ✓ El archivo `lib/utils.ts` existe con la función `cn()`
- ✓ Las dependencias Supabase, Zod y React Hook Form están instaladas (`npm list`)
- ✓ La estructura de carpetas (`auth/`), (`dashboard/`), `api/`, `components/`, `lib/`, `hooks/` está creada
- ✓ Las 5 tablas aparecen en Supabase Dashboard > Table Editor
- ✓ `next.config.ts` tiene los `remotePatterns` de Supabase configurados
- ✓ La página de prueba muestra el Card con botones en <http://localhost:3000>

Ejercicio Propuesto

Para reforzar lo aprendido, realiza las siguientes tareas por tu cuenta:

11. **Explora un componente:** Abre `components/ui/button.tsx` y lee el código. Identifica las variantes disponibles (`default`, `destructive`, `outline`, `secondary`, `ghost`, `link`). Prueba cada una en tu `page.tsx`.

12. **Agrega un Input:** Importa el componente Input de `@/components/ui/input` y agréalo dentro del Card. Dale un placeholder como "Ingresa tu email".
 13. **Inserta datos de prueba:** En Supabase Dashboard, ve a Table Editor > materias e inserta manualmente una materia de prueba (usa el botón Insert Row). Esto te ayudará a familiarizarte con la interfaz.
 14. **Investiga globals.css:** Abre `app/globals.css` y observa las variables CSS que agregó shadcn. Busca `--background`, `--foreground`, `--primary`. Cambia el valor de `--primary` y observa cómo afecta a los botones.
-

Próximo: Entrega 2A

En la siguiente entrega crearemos los clientes de Supabase para browser y servidor, configuraremos `proxy.ts` para la protección de rutas, y crearemos el callback de confirmación de email. Es la base de toda la autenticación.